

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 7. String and its Methods

Lecture: 78. Inquiry-Methods

Section 1: Lecture Summary

This lecture focused on Python string inquiry methods, which are string methods that return Boolean values — True or False — to answer questions about the contents or format of a string. The instructor introduced several inquiry methods such as `isalpha()`, `islower()`, `isupper()`, `istitle()`, `isspace()`, `isprintable()`, and `isidentifier()`. Through examples in a Jupyter notebook, each method was demonstrated by testing strings with different characteristics. The lecture explained what each method checks for and emphasized that these methods are very useful to verify or inquire string properties in programming. Additionally, the instructor explained important terms like whitespace characters (space, tab, newline, etc.), escape sequences (non-printable characters), and identifiers (valid variable names). The lecture concluded by encouraging students to practice these methods and recognizing their presence across other programming languages.

Section 2: Key Concepts and Explanations

Inquiry Methods Overview:

- Inquiry methods are string methods that return a Boolean value (True or False).
- They answer specific questions about string characteristics.

Specific Inquiry Methods and Their Meaning:

1. `isalpha()`

- Returns True if all characters in the string are alphabets (A-Z, a-z).
- Returns False if there are any numbers, special characters, or spaces.

2. islower()

- Returns True if all alphabetic characters are lowercase.
- Returns False if any character is uppercase.
- Non-alphabetic characters do not affect the result.

3. isupper()

- Returns True if all alphabetic characters are uppercase.
- Returns False otherwise.

4. istitle()

- Returns True if the string is in title case.
- Title case means each word's first letter is uppercase and the remaining letters are lowercase.
- If any word violates this, returns False.

5. isspace()

- Returns True if the string contains only whitespace characters.
- Whitespace includes spaces, tabs, newlines (`\n`), vertical tabs (`\v`), carriage returns (`\r`), form feeds (`\f`), etc.
- The string should not contain any other characters.
- An empty string returns False because it contains no characters.

6. isprintable()

- Returns True if all characters in the string are printable.
- Printable characters include letters, digits, punctuation, and whitespace.

- Returns False if the string contains any non-printable characters such as escape sequences (`\n`, `\v`, `\r`, `\b`, `alert \a`, escape character).

7. `isidentifier()`

- Returns True if the string is a valid Python identifier (variable name).
- Rules for valid identifiers:
 - Must start with a letter (a-z, A-Z) or underscore (`_`).
 - Can contain letters, digits (0-9), and underscores.
 - Cannot contain special characters or start with a digit.
 - Useful to verify if a string can be used as a variable name.

Additional Concepts:

- Whitespace characters help with formatting but do not produce visible characters. Includes spaces and control characters like newline (move cursor to new line), tab, carriage return.
- Escape sequences represent non-printable characters or actions in strings.
- Identifiers represent variable or function names chosen by the programmer. Keywords are reserved words in Python and not identifiers.

Section 3: Example Code and Use Cases

Example for isalpha()

Example for islower()

Example for isupper()

Example for istitle()

Example for isspace()

Example for isprintable()

Example for isidentifier()

```
s = 'Hello'
x = s.isalpha()
print(x) # Output: True because all are alphabets

s = 'Hello123'
x = s.isalpha()
print(x) # Output: False due to digits

s = 'hello'
x = s.islower()
print(x) # Output: True because all letters are lowercase

s = 'Hello'
x = s.islower()
print(x) # Output: False because of capital H

s = 'HELLO'
x = s.isupper()
print(x) # Output: True because all letters uppercase

s = 'Hello'
x = s.isupper()
print(x) # Output: False due to lowercase letters

s = 'Hello World'
x = s.istitle()
print(x) # Output: True - each word starts with uppercase, rest lowercase

s = 'Hello world'
x = s.istitle()
print(x) # Output: False - second word not title case

s1 = '         ' # seven spaces
```

```
print(len(s1)) # Output: 7
print(s1.isspace()) # Output: True

s2 = '' # empty string
print(len(s2)) # Output: 0
print(s2.isspace()) # Output: False

s3 = '\n\t ' # new line, tab, space (all whitespace)
print(s3.isspace()) # Output: True

s4 = ' a ' # contains non-space character
print(s4.isspace()) # Output: False

s = 'Hello World'
print(s.isprintable()) # Output: True (all printable characters)

s = 'Hello\nWorld'
print(s.isprintable()) # Output: False (contains newline escape sequence)

s = 'item1'
print(s.isidentifier()) # Output: True (valid variable name)

s = '1item'
print(s.isidentifier()) # Output: False (starts with digit)

s = 'item_1'
print(s.isidentifier()) # Output: True (underscore allowed)

s = 'item?'
print(s.isidentifier()) # Output: False (special character not allowed)
```

Each example demonstrates how the respective inquiry method can be used to check string properties and validate data before further processing.

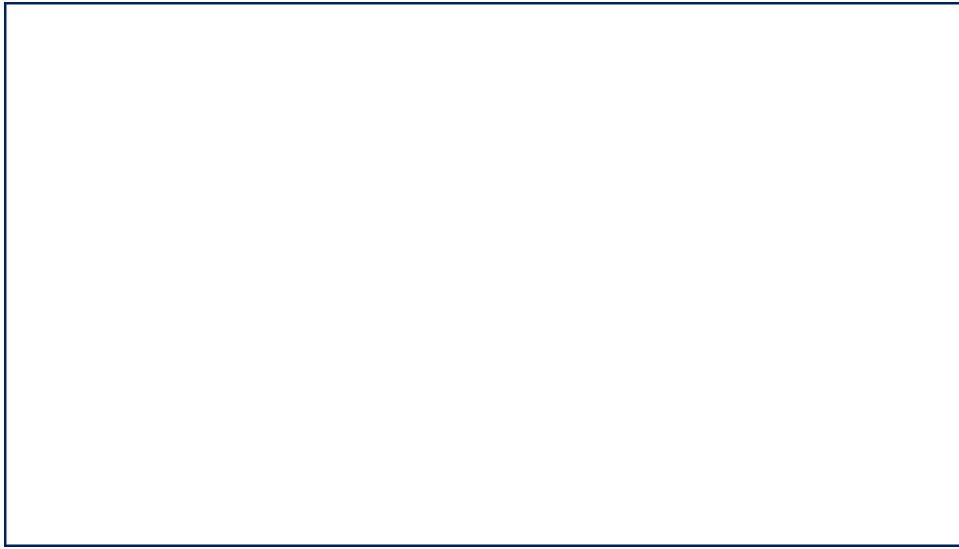
Section 4: Key Takeaways

- Inquiry methods return Boolean (True/False) indicating specific string properties.
- `isalpha()` verifies if a string contains only alphabetic characters.
- `islower()`, `isupper()`, and `istitle()` check letter casing patterns in strings.

- `isspace()` tests if the string contains only whitespace characters (spaces, tabs, newlines, etc.), with empty strings returning `False`.
- `isprintable()` confirms if all characters in the string produce visible output (excludes escape sequences).
- `isidentifier()` checks if a string is a valid Python variable name according to language rules.
- These methods do not take parameters and provide quick checks useful in data validation, parsing, or formatting tasks.
- Understanding whitespace and escape characters helps in effectively using these methods.
- Learning these string inquiry methods in Python also prepares learners for similar functionality in other programming languages.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Inquiry Methods



Inquiry Methods

String Methods

Inquiry Methods
String Methods

Inquiry Methods
String Methods

Boolean

String Methods

True / False

String Methods

True / False

isalpha ()

String Method

True / False

islower ()

String Methods

True / False

islower ()

isupper ()

String Method

True / False

istitle ()

Inquiry Methods

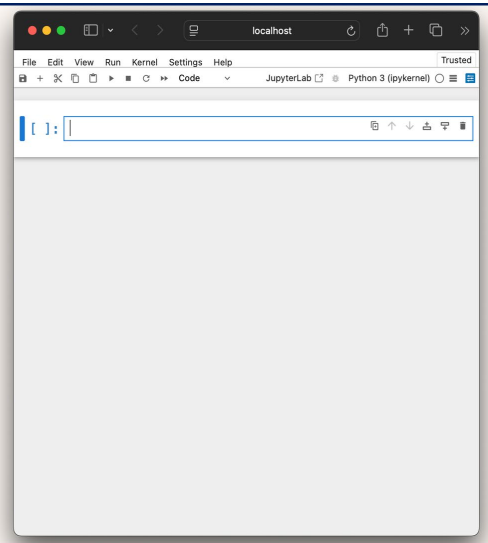
Using strings

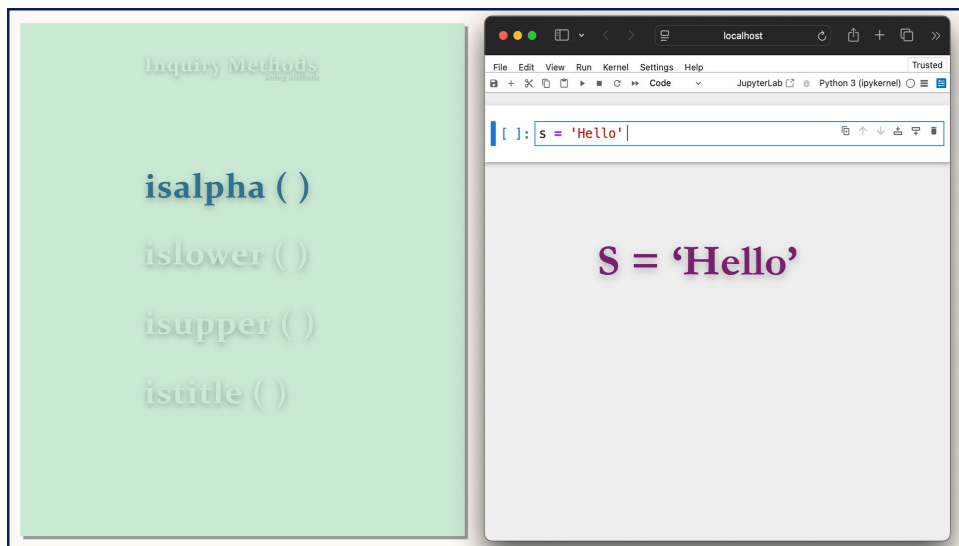
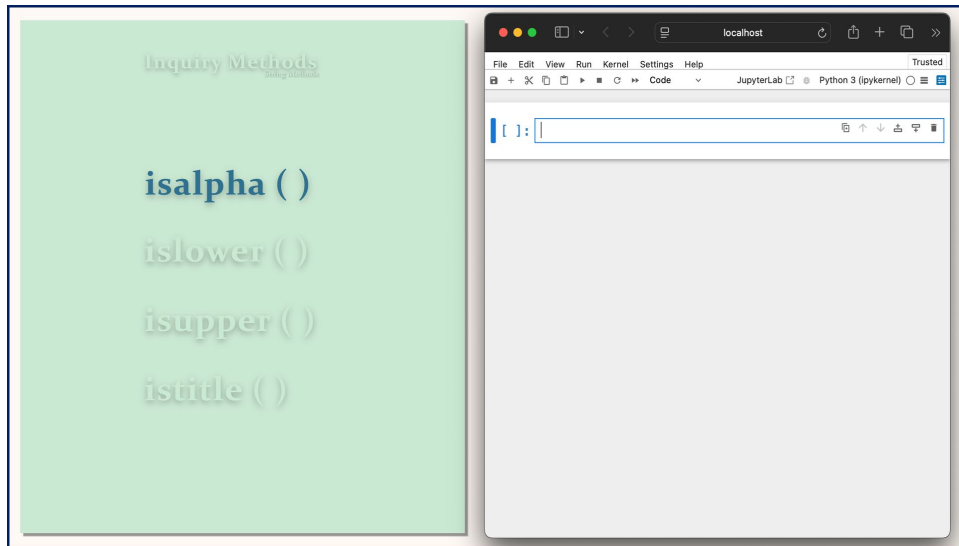
`isalpha ( )`

`islower ( )`

`isupper ( )`

`istitle ( )`





Inquiry Methods
asking questions

isalpha ()

islower ()

isupper ()

istitle ()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = 'Hello'
```

```
[ ]:
```

S

0	1	2	3	4
H	e	l	l	o

Inquiry Methods
asking questions

isalpha ()

islower ()

isupper ()

istitle ()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = 'Hello'
```

```
[ ]:
```

S

0	1	2	3	4
H	e	l	l	o

x = s.isalpha()

Inquiry Methods
String Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
```

```
[ ]: x = s.isalpha()
```

	0	1	2	3	4
s	H	e	l	l	o

x = s.isalpha()

Inquiry Methods
String Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
```

```
[2]: x = s.isalpha()
```

```
[ ]:
```

	0	1	2	3	4
s	H	e	l	l	o

True x = s.isalpha()

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
[2]: x = s.isalpha()
[ ]: print(x)
```

	0	1	2	3	4
s	H	e	l	l	o

True x = s.isalpha()

Inquiry Methods

isalpha ()

islower ()

isupper ()

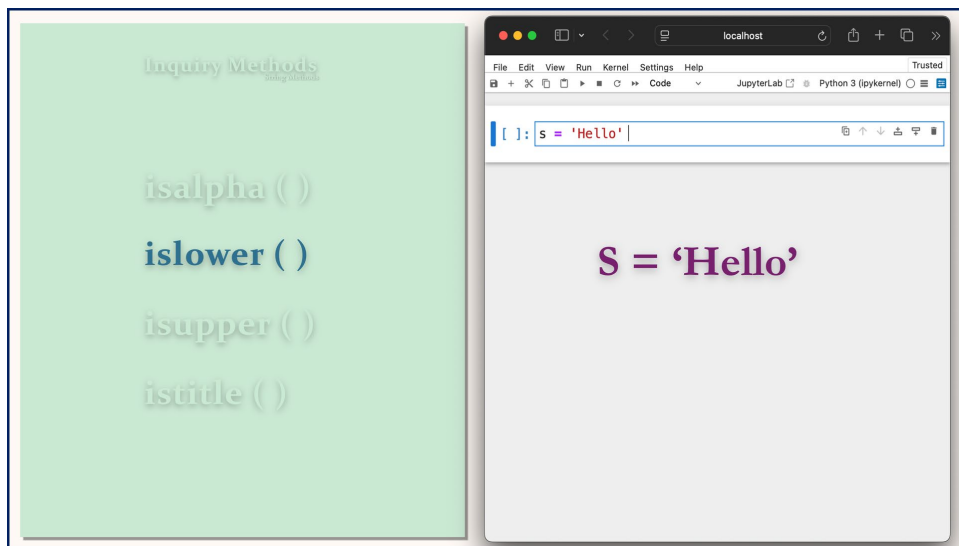
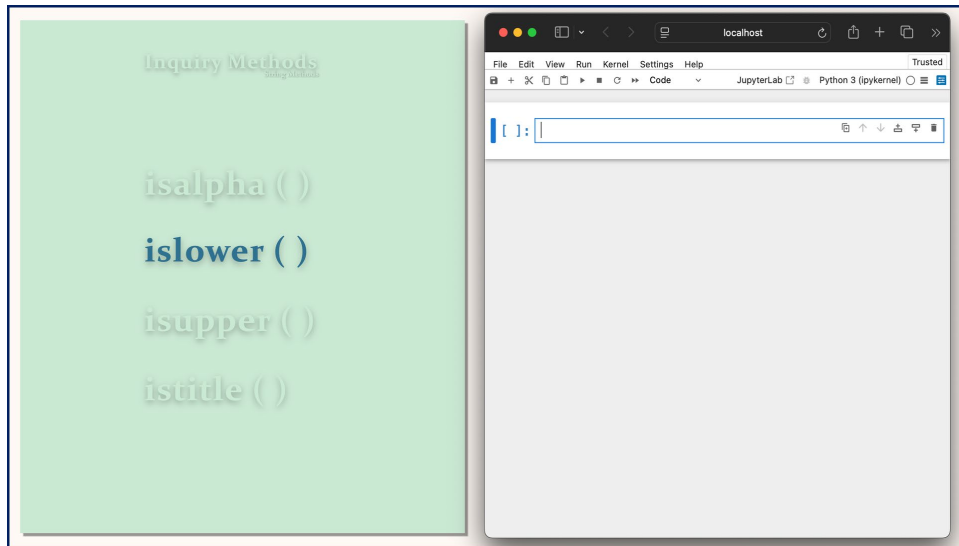
istitle ()

```
[1]: s = 'Hello'
[2]: x = s.isalpha()
[3]: print(x)
```

True

	0	1	2	3	4
s	H	e	l	l	o

True x = s.isalpha()



Inquiry Methods

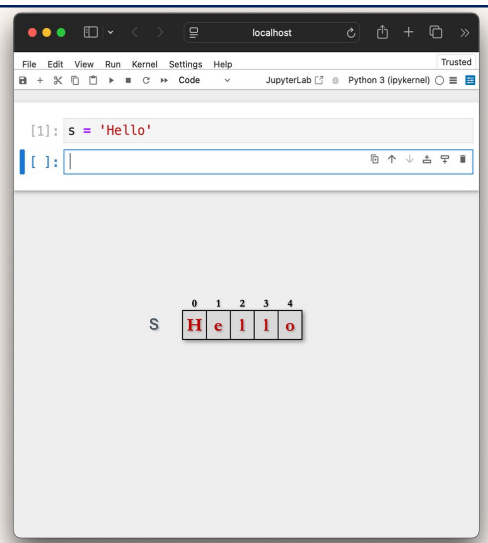
string attributes

isalpha ()

islower ()

isupper ()

istitle ()



```
[1]: s = 'Hello'
```

```
[ ]:
```

s

	0	1	2	3	4
	H	e	l	l	o

Inquiry Methods
using strutils

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
```

```
[ ]:
```

	0	1	2	3	4
s	H	e	l	l	o

```
x = s.islower( )
```

Inquiry Methods
using strutils

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
```

```
[ ]: x = s.islower()
```

	0	1	2	3	4
s	H	e	l	l	o

```
x = s.islower( )
```

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
[2]: x = s.islower()
[ ]:
```

	0	1	2	3	4
s	H	e	l	l	o

False **x = s.islower()**

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
[2]: x = s.islower()
[ ]: print(x)
```

	0	1	2	3	4
s	H	e	l	l	o

False **x = s.islower()**

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello'
[2]: x = s.islower()
[3]: print(x)
False
[ ]:
```

	0	1	2	3	4
s	H	e	l	l	o

False x = s.islower()

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[ ]: s = 'hello'
```

S = 'hello'

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'hello'
```

```
[ ]:
```

	0	1	2	3	4
s	h	e	l	l	o

```
x = s.islower( )
```

Inquiry Methods

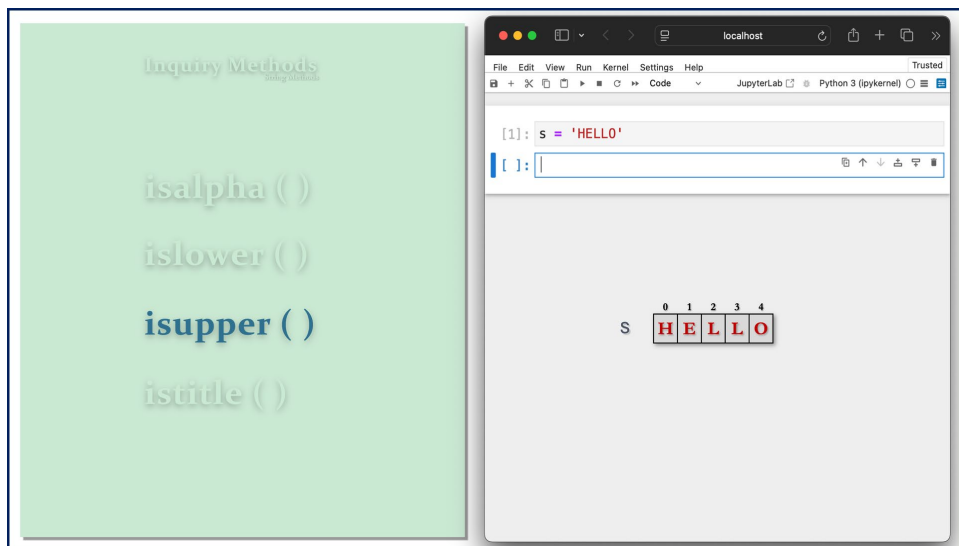
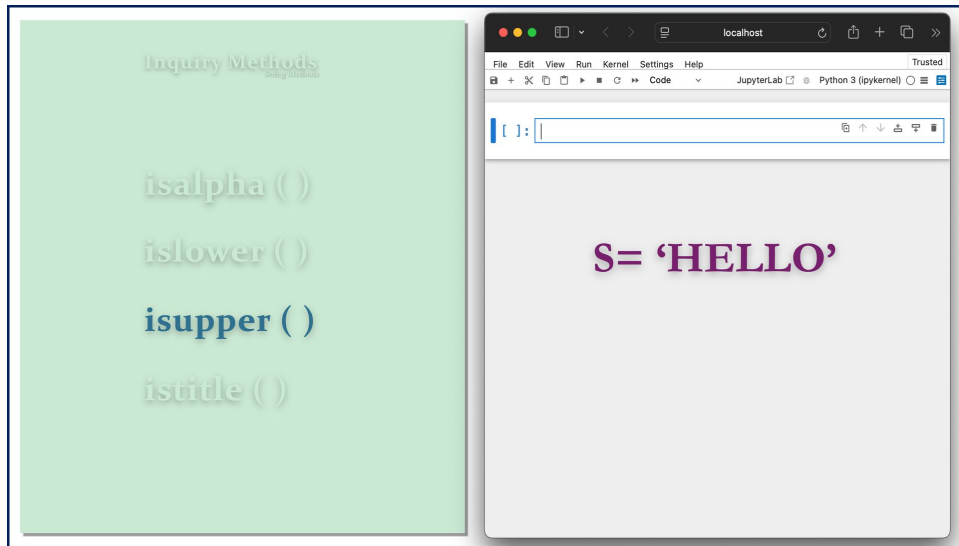
isalpha ()

islower ()

isupper ()

istitle ()

```
[ ]:
```



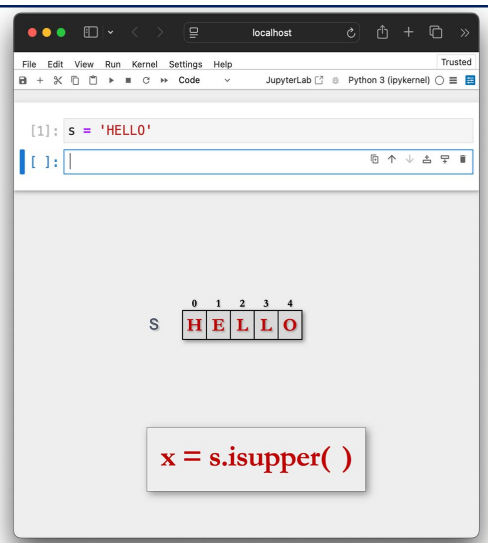
Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()



The screenshot shows a JupyterLab window with a code cell containing the following Python code:

```
[1]: s = 'HELLO'
```

The output of the code cell is displayed below the code, showing the variable `s` and its value `'HELLO'` in a table format:

	0	1	2	3	4
s	H	E	L	L	O

Below the output table, there is a code cell containing the following Python code:

```
x = s.isupper( )
```

Inquiry Methods
string attributes

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'HELLO'
```

```
[2]: x = s.isupper()
```

	0	1	2	3	4
s	H	E	L	L	O

x = s.isupper()

Inquiry Methods
string attributes

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'HELLO'
```

```
[2]: x = s.isupper()
```

```
[ ]:
```

	0	1	2	3	4
s	H	E	L	L	O

True **x = s.isupper()**

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'HELLO'
[2]: x = s.isupper()
[3]: print(x)
```

S

0	1	2	3	4
H	E	L	L	O

True **x = s.isupper()**

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

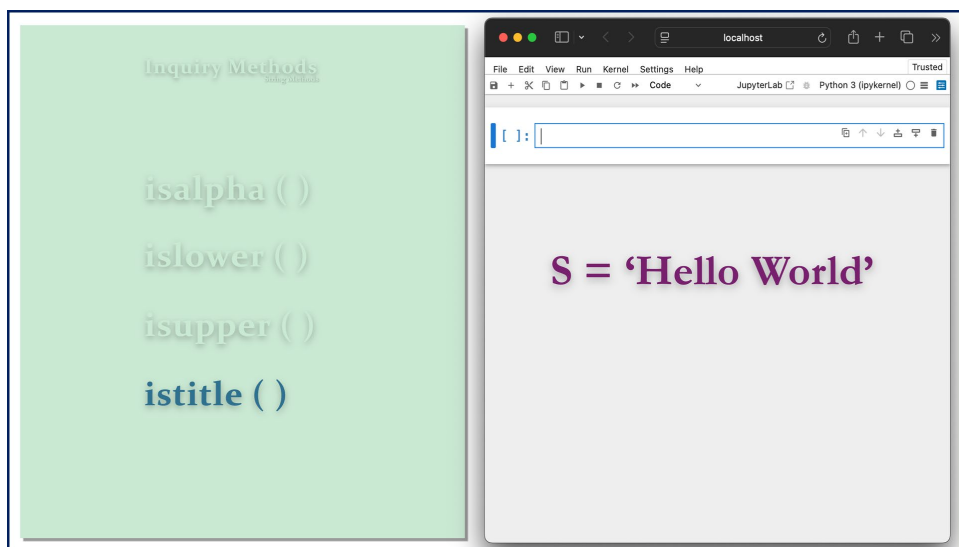
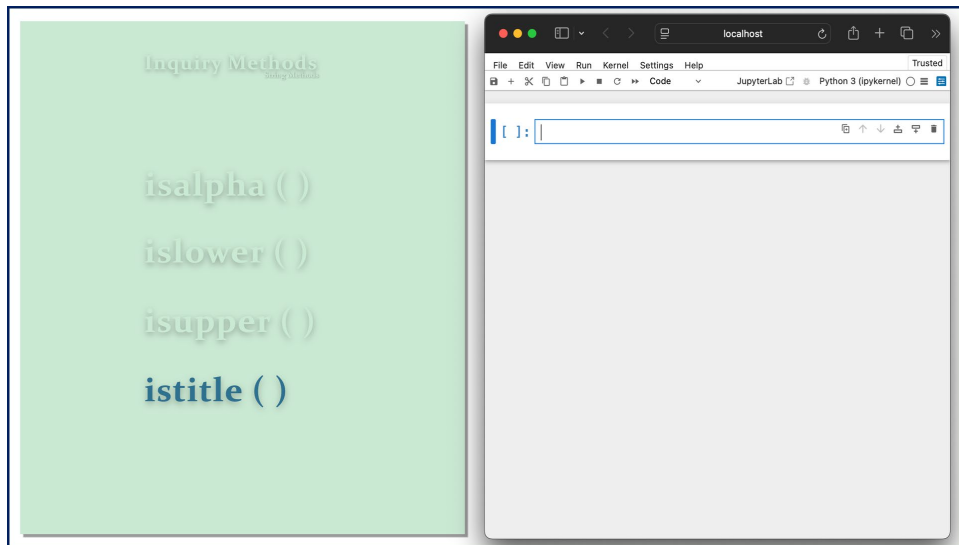
```
[1]: s = 'HELLO'
[2]: x = s.isupper()
[3]: print(x)
```

True

S

0	1	2	3	4
H	E	L	L	O

True **x = s.isupper()**



Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello World'
```

```
[ ]:
```

0	1	2	3	4	5	6	7	8	9	10	
S	H	e	l	l	o		W	o	r	l	d

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello World'
```

```
[ ]:
```

0	1	2	3	4	5	6	7	8	9	10	
S	H	e	l	l	o		W	o	r	l	d

```
x = s.istitle( )
```

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

The JupyterLab interface displays the following code in the first two cells:

```
[1]: s = 'Hello World'
[2]: x = s.istitle()
```

The third cell is currently empty, with the cursor at the start of the line `[ ]:`.

Below the code cells, a string visualization shows the string `s` as `'Hello World'` with indices 0 through 10 above each character.

At the bottom, a box displays the result of the `istitle()` method: `True` and `x = s.istitle( )`.

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

The JupyterLab interface displays the following code in the first two cells:

```
[1]: s = 'Hello World'
[2]: x = s.istitle()
```

The third cell contains the code `print(x)`.

Below the code cells, a string visualization shows the string `s` as `'Hello World'` with indices 0 through 10 above each character.

At the bottom, a box displays the result of the `istitle()` method: `True` and `x = s.istitle( )`.

Inquiry Methods

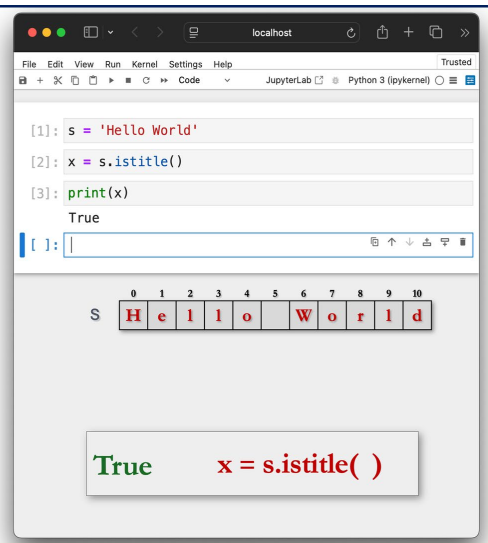
Using attributes

`isalpha ( )`

`islower ( )`

`isupper ( )`

`istitle ( )`



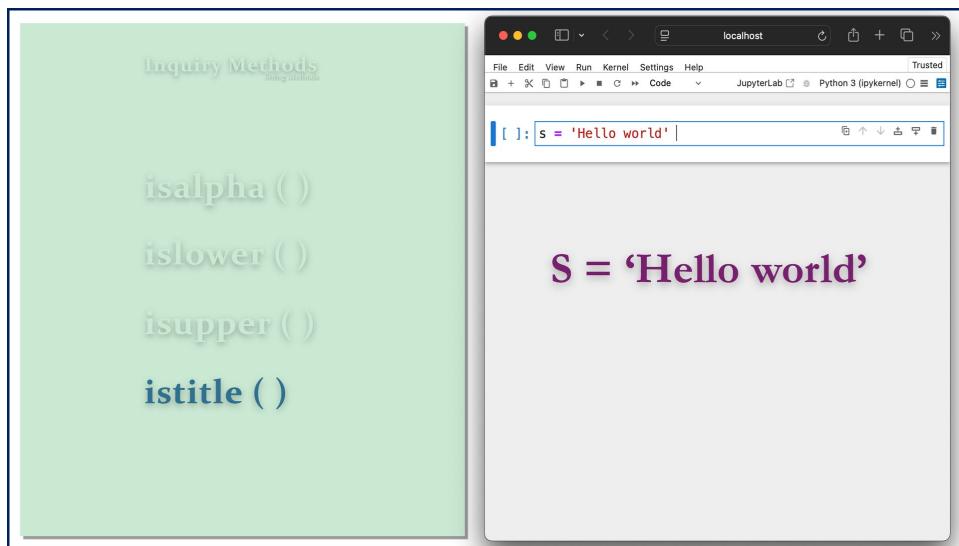
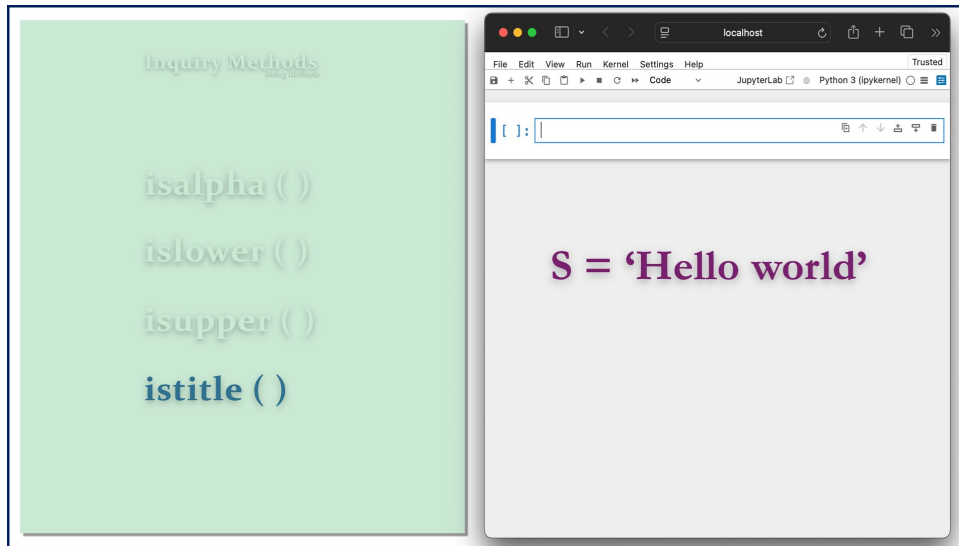
The screenshot shows a JupyterLab window with the following code and output:

```
[1]: s = 'Hello World'
[2]: x = s.istitle()
[3]: print(x)
True
```

The output is displayed as a table with indices 0 through 10 above the characters of the string 'Hello World':

	0	1	2	3	4	5	6	7	8	9	10
S	H	e	l	l	o		W	o	r	l	d

Below the table, the output is shown as a box containing the text: `True` `x = s.istitle( )`



Inquiry Methods
using strutils

isalpha ()

islower ()

isupper ()

istitle ()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = 'Hello world'
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10
s	H	e	l	l	o		w	o	r	l	d

Inquiry Methods
using strutils

isalpha ()

islower ()

isupper ()

istitle ()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = 'Hello world'
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10
s	H	e	l	l	o		w	o	r	l	d

x = s.istitle ()

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello world'
```

```
[ ]: x = s.istitle()
```

	0	1	2	3	4	5	6	7	8	9	10
s	H	e	l	l	o		w	o	r	l	d

x = s.istitle ()

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello world'
```

```
[2]: x = s.istitle()
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10
s	H	e	l	l	o		w	o	r	l	d

False **x = s.istitle ()**

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello world'
[2]: x = s.istitle()
[3]: print(x)
```

0	1	2	3	4	5	6	7	8	9	10
S	H	e	l	l	o	w	o	r	l	d

False **x = s.istitle()**

Inquiry Methods

isalpha ()

islower ()

isupper ()

istitle ()

```
[1]: s = 'Hello world'
[2]: x = s.istitle()
[3]: print(x)
```

False

0	1	2	3	4	5	6	7	8	9	10
S	H	e	l	l	o	w	o	r	l	d

False **x = s.istitle()**

Inquiry Methods

String Methods

isspace ()

Inquiry Methods

String Methods

isspace ()

isprintable ()

Inquiry Methods

String Methods

isspace ()

isprintable ()

isidentifier ()

Inquiry Methods

`isspace ( )`

`isprintable ( )`

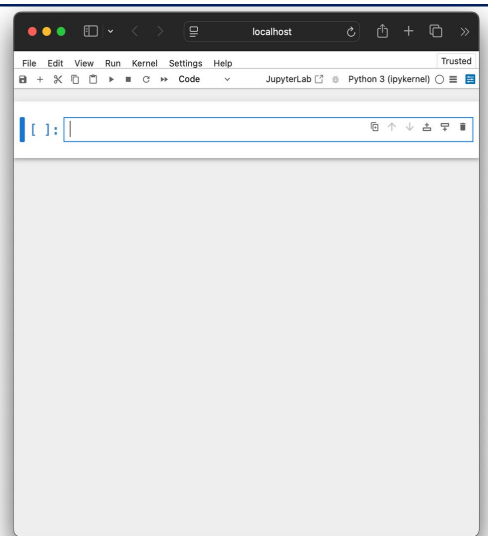
`isidentifier ( )`

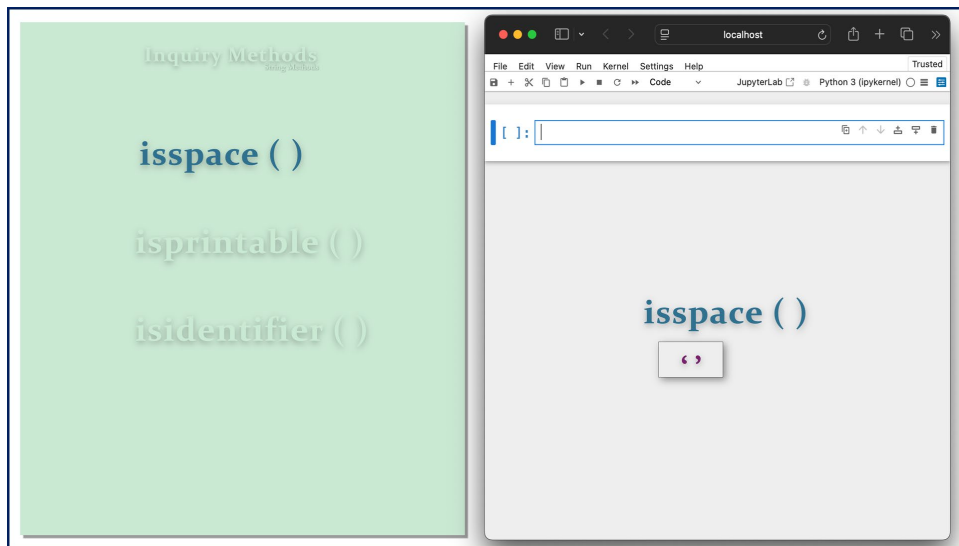
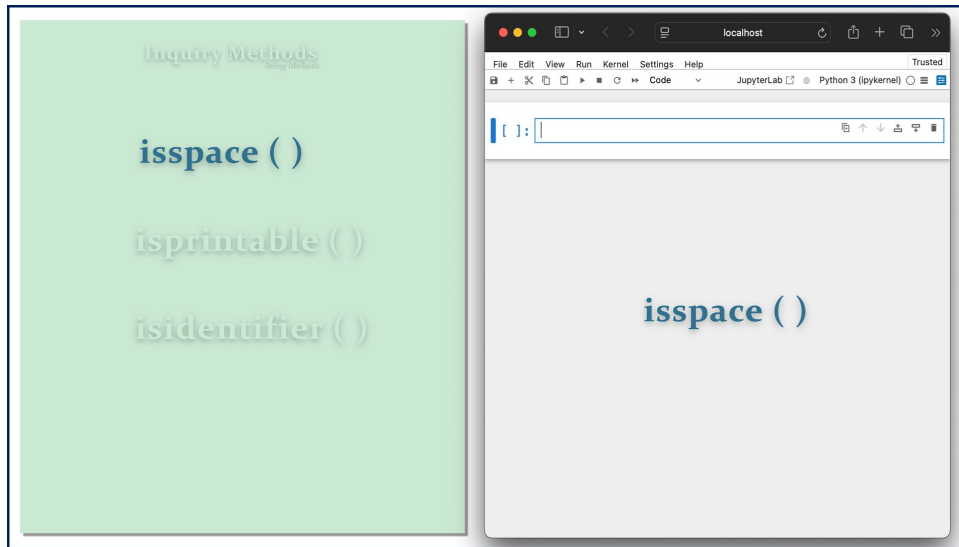
Inquiry Methods

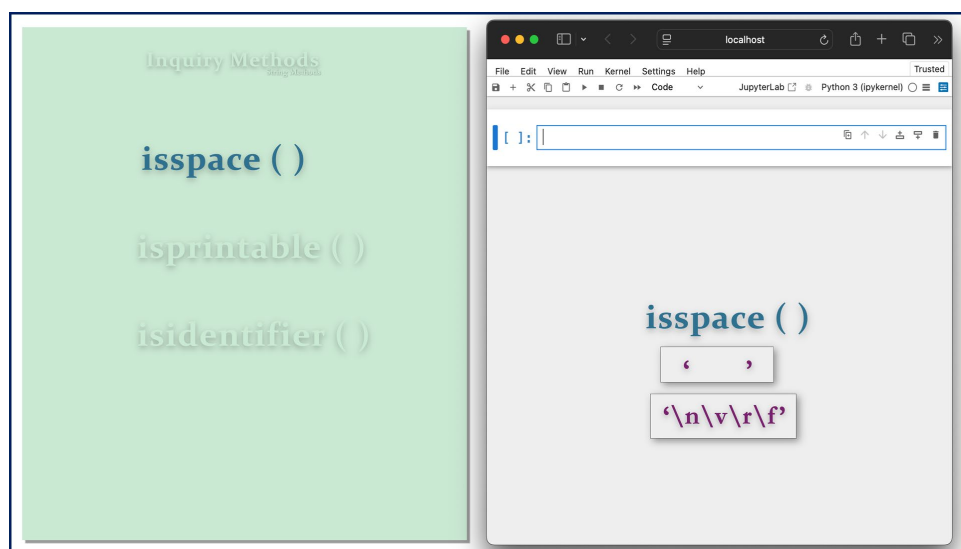
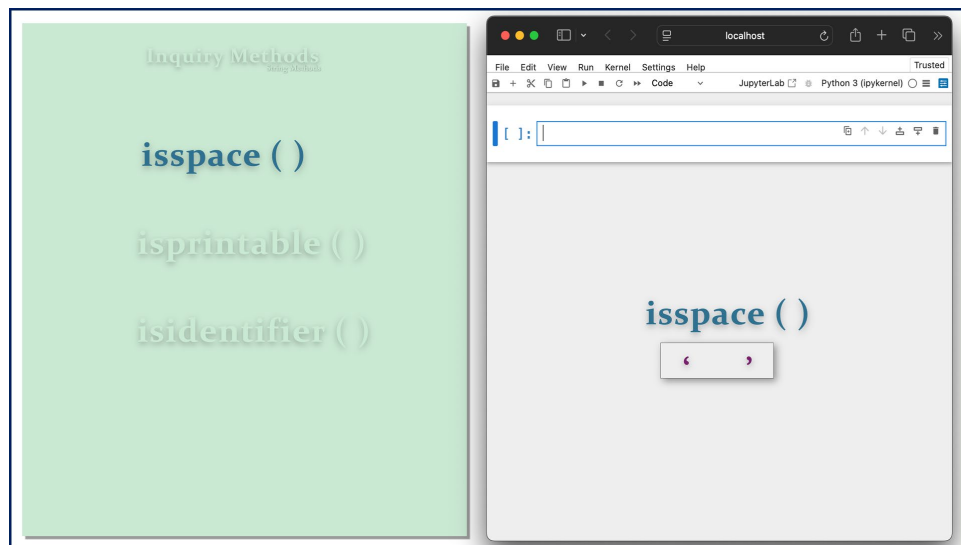
`isspace ( )`

`isprintable ( )`

`isidentifier ( )`







Inquiry Methods

`isspace ( )`

`isprintable ( )`

`isidentifier ( )`

`isspace ( )`

`' '`

`'\n\v\r\f'`

White Space

Inquiry Methods

`isspace ( )`

`isprintable ( )`

`isidentifier ( )`

`isspace ( )`

`' '`

`'\n\v\r\f'`

White Space

`'\t'`

Inquiry Methods
String Methods

isspace ()

isprintable ()

isidentifier ()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[]:

isspace ()

‘ ’

‘\n\v\r\f’

} White Space

‘’ Empty String

Inquiry Methods
String Methods

isspace ()

isprintable ()

isidentifier ()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[]:

isspace ()

‘ ’

‘\n\v\r\f’

} White Space

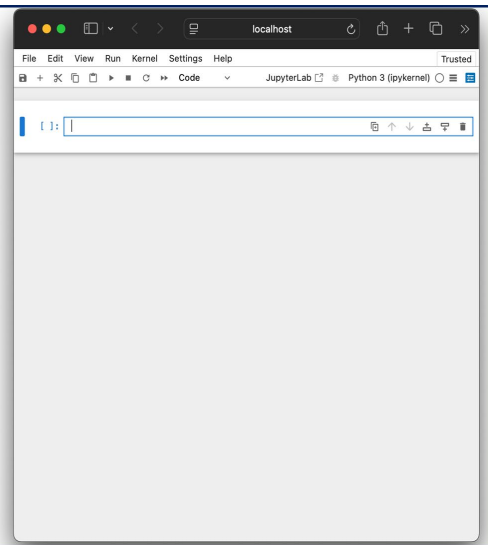
✗ ‘’ Empty String

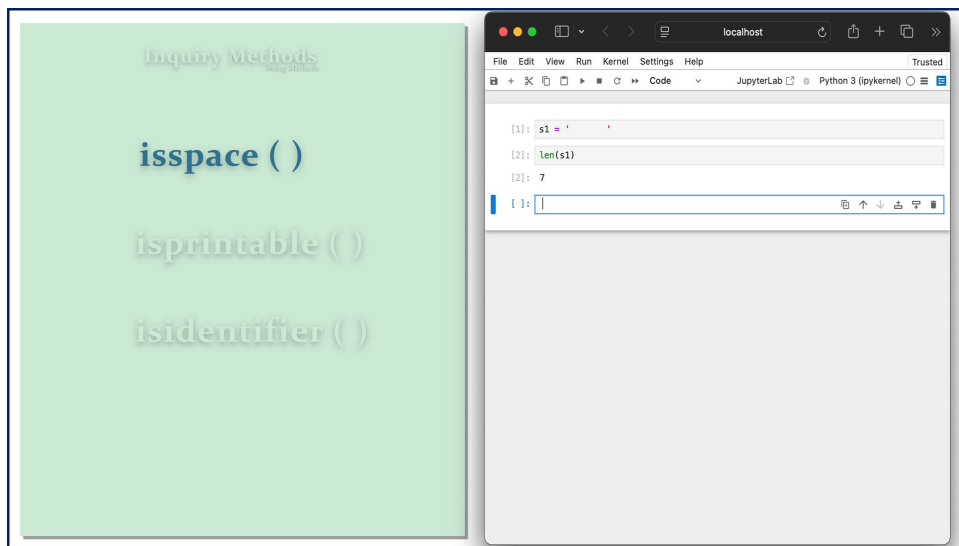
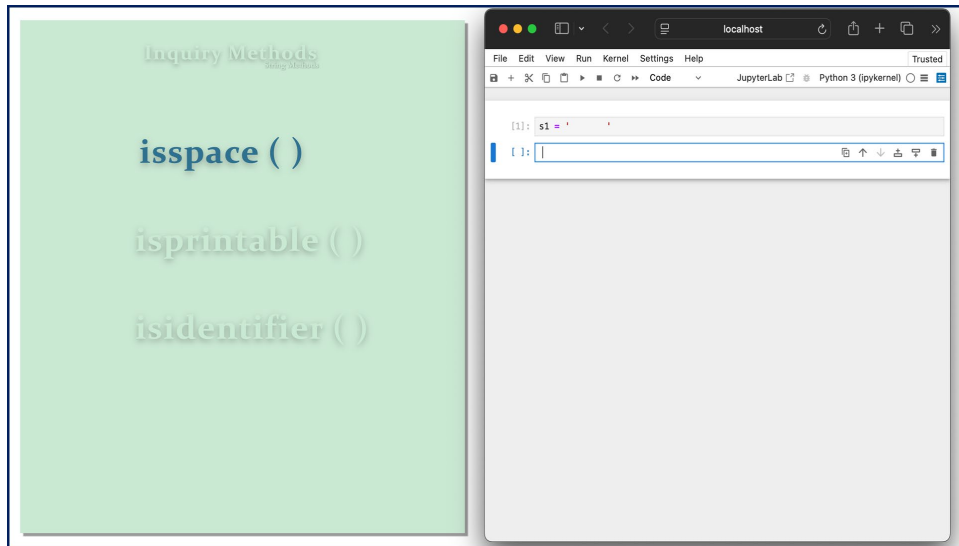
Inquiry Methods
Using Methods

isspace ()

isprintable ()

isidentifier ()





Inquiry Methods
String Methods

isspace ()

isprintable ()

isidentifier ()

```
[1]: s1 = " "
[2]: len(s1)
[2]: 7
[3]: print(s1.isspace())
True
[ ]:
```

Inquiry Methods
String Methods

isspace ()

isprintable ()

isidentifier ()

```
[1]: s1 = " "
[2]: len(s1)
[2]: 7
[3]: print(s1.isspace())
True
[4]: s2 = ""
[ ]:
```

Inquiry Methods
Using JupyterLab

isspace ()

isprintable ()

isidentifier ()

```
[1]: s1 = ' '
[2]: len(s1)
[2]: 7
[3]: print(s1.isspace())
True
[4]: s2 = ''
[5]: len(s2)
[5]: 0
[6]:
```

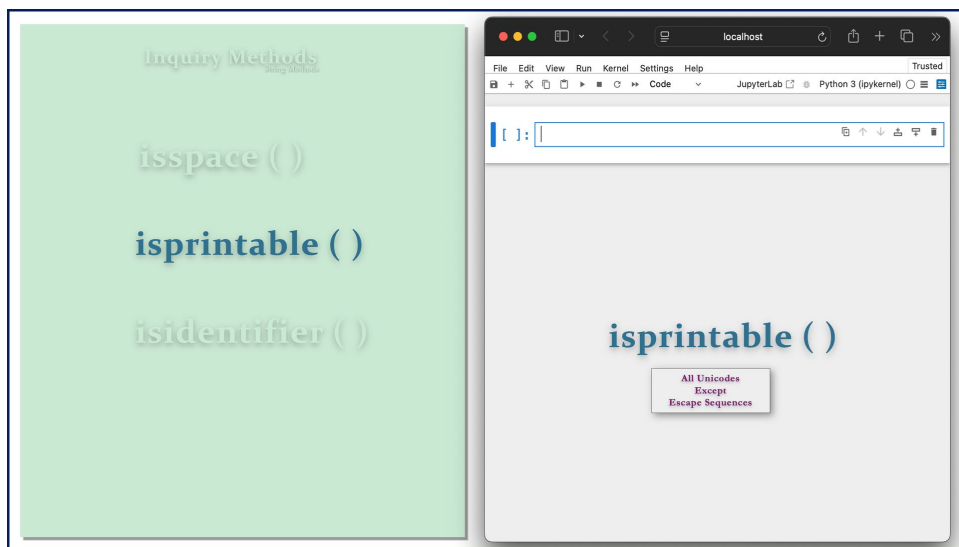
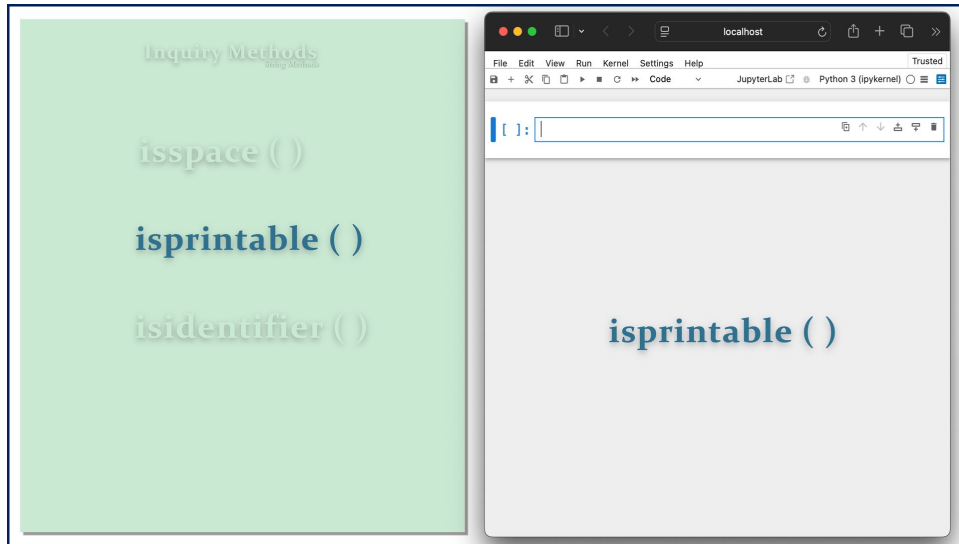
Inquiry Methods
Using JupyterLab

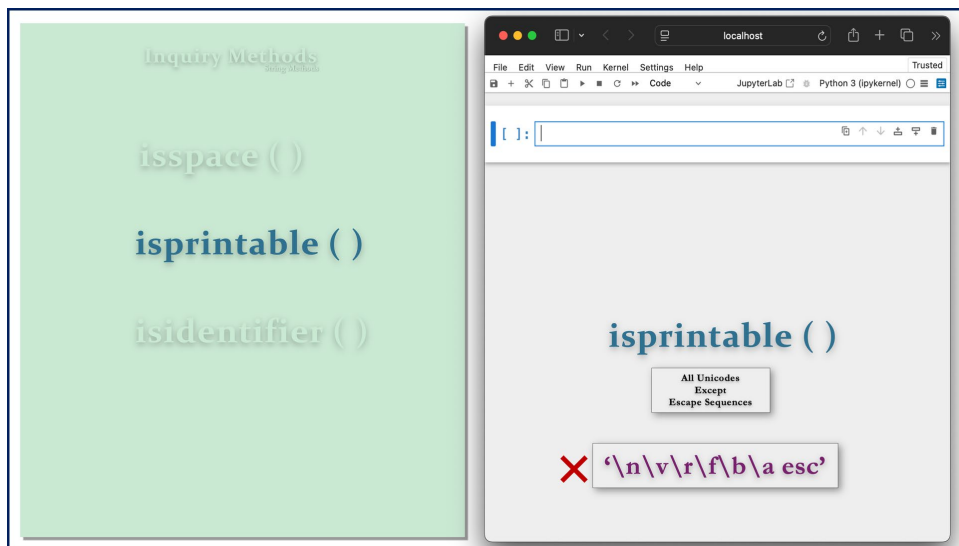
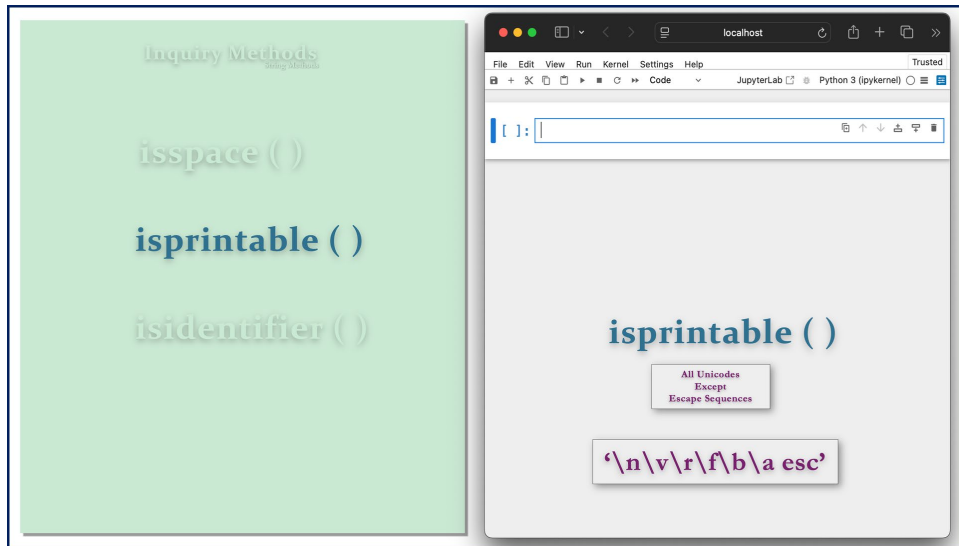
isspace ()

isprintable ()

isidentifier ()

```
[1]: s1 = ' '
[2]: len(s1)
[2]: 7
[3]: print(s1.isspace())
True
[4]: s2 = ''
[5]: len(s2)
[5]: 0
[6]: print(s2.isspace())
False
[7]:
```





Inquiry Methods

`isspace ( )`

`isprintable ( )`

`isidentifier ( )`

`isidentifier ( )`

